

Aquí se pueden ver algunos programas en los que estuve trabajando para entender el correcto funcionamiento de la ESP

Nota. Sigo con la preocupación de si esta bien lo que hace mi ESP, ya que en la Terminal de Thonny Micropython no me deja salir y tengo que forzarla a salir para poder modificar el código, no estoy seguro si es por algo que este omitiendo o si así es como funciona normalmente o en el peor de los casos no se si la ESP este dañada

1. El primer código “[## Código fusilado de un tipo que hizo un conador con deepsleep](#)” fue para entender como es el funcionamiento de la función Deepsleep, ya que fue tras varios intentos no se podía llevar a lo requerido, así que se tuvo que copiar el codigo de un video en donde hacen uso del Deepsleep, la diferencia aquí es que se tuvo que pasar el código original de C+ a Python y verificar que funcionara. Dado que funciona muy bien se continuo usando el mismo método.

Anexo link: [Beyond the Zzz's: ESP32's External Wake-Up in Deep Sleep Explored](#)

2. Luego tenemos “[## Deepsleep con contador](#)”, teniendo ya el correcto funcionamiento del Deepsleep, se acopló una “tarea” en este caso un contador de 10 a 0 que al momento de terminar, este entra automáticamente en Deepsleep, y la ESP puede ser despertada por un botón o esperando 5 seg
3. Para el caso de “[## Deepsleep guardando un dato](#)” se implementa casi el mismo mecanismo del contador, solo que ahora inicia en 10, luego entra en deepsleep, después despierta y busca si hay algún dato guardado; en este caso tiene un 9 y lo imprime, para después volver a dormirse y repetir el ciclo, una vez haya llegado a 0, se detiene y ya no entra en deepsleep. Este programa se diseñó con la intención de ver si el dato que guarda se sobrescribe en la memoria RTC o si va guardando varios datos, y por lo que parece solo lo sobrescribe.
4. Ahora implementamos la interrupción, bandera, handler o ISR (interrupt service routine) “[## DeepSleep + Interrupción](#)” por lo que se agrega otro botón que interrumpe el contador, entra en Deepsleep y por último después de 5 segundos despierta y vuelve a contar, este contador puede ser interrumpido por el Handler para entrar de forma rápida en modo Deepsleep de

forma indefinida y con el otro botón se puede despertar incluso antes de los 5 segundos del otro.

5. Por último se tiene el “[## Deepsleep + DHT11 + RTC para obtener la hora](#)” en este se toma todo lo anterior además de el proyecto DHT11, ahora el programa puede dar una hora local, colocada inicialmente y cada que despierta dice el dato de temperatura y humedad con la fecha establecida, para después entrar en Deepsleep, co ayuda de un botón vuelve a despertar y a dar la nueva medición de temperatura y humedad además de la hora y ese mismo momento de medición por lo que se podría llevar un registro sencillo

```
## Código fusilado de un tipo que hizo un conador con deepsleep
from machine import Pin, deepsleep, RTC
import esp32
import time

# Configuración inicial
wakeup_pin = Pin(2, Pin.IN, Pin.PULL_UP) # GPIO 2 como entrada con PULL_UP
rtc = RTC()

# Leer o inicializar el contador de reinicios en memoria RTC
boot_count = rtc.memory()
if boot_count:
    boot_count = int(boot_count.decode()) + 1
else:
    boot_count = 1

# Mostrar el número de reinicios
print(f"Boot number: {boot_count}")

# Guardar el contador actualizado en la memoria RTC
rtc.memory(str(boot_count).encode())

# Configurar el pin para despertar del deep sleep
esp32.wake_on_ext0(pin=wakeup_pin, level=esp32.WAKEUP_ALL_LOW)

# Agregar un retardo de 1 segundo para evitar activaciones múltiples
time.sleep(1)

# Mensaje antes de entrar en deep sleep
print("I'm going to sleep now.")

# Entrar en deep sleep
```

```
deepsleep()
```

Deepsleep con contador

```
import esp32
from machine import Pin, deepsleep
from time import sleep

## Configuración inicial
wake = Pin(14, Pin.IN, Pin.PULL_DOWN)

i = 10

## Configurando el pin para despertar
esp32.wake_on_ext0(pin=wake, level=esp32.WAKEUP_ALL_DOWN)

## Este código se ejecutará cuando el ESP32 despierte
print('La ESP-32 ha despertado... Holi :3')

## Código principal: la cuenta regresiva
while i > 0:
    print('Preparando para dormir en', i)
    i -= 1
    sleep(1) # Espera 1 segundo

else:
    print('Entrando en deep sleep.....')
    deepsleep(5000) # Duerme por 5 segundos ó despierta con el pin
```

Deepsleep guardando un dato

```
import esp32
from machine import Pin, deepsleep, RTC
from time import sleep

## Configuración inicial
wake = Pin(14, Pin.IN, Pin.PULL_UP)
rtc = RTC()

## Verifica si hay datos en la memoria RTC
i = rtc.memory()
if i:
    i = int(i) ## Recupera el valor guardado
else:
    i = 10 ## Valor inicial si no hay datos
```

```
## Este código se ejecutará cuando el ESP32 despierte
print('La ESP-32 ha despertado... Holi :3')

## Configura el pin para despertar
esp32.wake_on_ext0(pin=wake, level=esp32.WAKEUP_ALL_LOW)

## tarea principal
if i > 0:
    print('nos quedamos en', i)
    i -= 1
    rtc.memory(str(i).encode()) ## Guarda el valor actualizado en memoria RTC
    sleep(1) ## Espera 1 segundo
    print('Entrando en deep sleep.....')
    deepsleep(5000) ## Duerme por 5 segundos
else:
    print('Contador terminado, no se volverá a dormir.')
```

DeepSleep + Interrupcion

```
import esp32
from machine import Pin, deepsleep
from time import sleep

## Configuración de los botones
interrupt_button = Pin(2, Pin.IN, Pin.PULL_UP) ## Botón para entrar en deep sleep indefinido
wake_button = Pin(4, Pin.IN, Pin.PULL_UP) ## Botón para despertar

## Configuración del pin para despertar mientras esta en DeepSleep
esp32.wake_on_ext0(pin=wake_button, level=esp32.WAKEUP_ALL_LOW)

## Función de interrupción: entra en deep sleep indefinido
def enter_deep_sleep(pin):
    print("Interrupción activada. Entrando en deep sleep indefinido.....")
    deepsleep() ## Deep sleep indefinido

## Configurar la interrupción en el botón de deep sleep
interrupt_button.irq(trigger=Pin.IRQ_RISING, handler=enter_deep_sleep)

## Inicio del programa
print("La ESP-32 ha despertado... Holi :3")

## Contador descendente
for i in range(10, 0, -1):
```

```
print("Contando: ",i)
sleep(1) ## Espera 1 segundo entre números

# Mensaje antes de entrar en deep sleep indefinido
print("La cuenta terminó. Entrando en deep sleep de 5 segundos")
deepsleep(5000)
```

```
## Deepsleep + DHT11 + RTC para obtener la hora
import esp32
from machine import Pin, deepsleep, RTC
from time import sleep, localtime, mktime
import dht

## Botón para despertar
wake_button = Pin(4, Pin.IN, Pin.PULL_UP)

## Configuración del pin para despertar desde deepsleep
esp32.wake_on_ext0(pin=wake_button, level=esp32.WAKEUP_ALL_LOW)

## Inicializamos el sensor DHT11
sensor = dht.DHT11(Pin(15))

# RTC para mantener el tiempo entre deep sleep
rtc = RTC()

# Configuración inicial del RTC si es la primera vez
if not rtc.datetime()[0]: # Si el RTC no tiene tiempo configurado
    # Configura la hora inicial (año, mes, día, hora, minuto, segundo, día de la semana, subsegundos)
    rtc.datetime((2024, 12, 16, 1, 0, 0, 0, 0)) # Inicializa el RTC con una fecha/hora específica

# Recupera la hora actual
current_time = localtime() # Devuelve la hora en formato (año, mes, día, hora, minuto, segundo,
día_semana, día_año)

# Imprime la hora de despertar
print("-----")
print("La ESP-32 ha despertado... Holi :3")
print("Hora actual: {:04d}-{:02d}-{:02d} {:02d}:{:02d}:{:02d}".format(
    current_time[0], current_time[1], current_time[2], # Año, mes, día
    current_time[3], current_time[4], current_time[5] # Hora, minuto, segundo
))

# Toma una medición del sensor DHT11
try:
    sensor.measure()
```

```
temp = sensor.temperature()
hum = sensor.humidity()

print("Temperatura: {}°C  Humedad: {}%".format(temp, hum))

except OSError as e:
    print("Error al leer el sensor, conectalo bien!", e)

# Actualiza la hora sumando el tiempo de deep sleep (si es necesario)
# Aquí no sumamos tiempo porque es indefinido, pero si tienes un tiempo fijo puedes hacerlo.

# Mensaje antes de entrar en deep sleep indefinido
print("Medición completada. Entrando en deepsleep...")
print("-----")
deepsleep()
```